

Explorando la comunicación RS232

Comunicaciones Digitales UMNG

jose.rugeles@unimilitar.edu.co

1. OBJETIVOS

- Comprobar la relación de tiempo de bit / tasa de transmisión de datos.
- Analizar los detalles de la capa física del estándar RS 232.
- Analizar la estructura del protocolo de comunicación serial, identificando bit de inicio, bit de parada, datos y paridades.
- Desarrollar habilidades de programación empleando Micropython.

2. COMPETENCIAS A DESARROLLAR

El estudiante deberá tener dominio de al menos los siguientes conceptos y herramientas así:

- Capacidad en el manejo del osciloscopio digital para la medida de señales transitorias.
- Medición del tiempo de bit en una señal digital.
- Análisis de las señales digitales al modificar el parámetro paridad y tasa de baudios.
- Establecer la relación tiempo de bit y tasa de baudios.
- Analizar una señal digital respecto al tiempo.
- Habilidad para interconectar dispositivos mediante el protocolo de comunicación.
- Desarrollo de algoritmos de comunicación empleando MicroPython.

3. PROCEDIMIENTO

Empleando un dispositivo Raspberry Pi Pico 2W:

3.1. Realice la configuración inicial del dispositivo

Descargue el Firmware en: <https://www.raspberrypi.com/documentation/microcontrollers/micropython.html>

Siga las instrucciones descritas en esta página para descargar el archivo .UF2 correspondiente al dispositivo.

3.2. Identifique los terminales TX y RX en las UART del dispositivo

(1 y 2 para el UART0). Conecte un osciloscopio digital entre la terminal TX y tierra del dispositivo (Consulte las especificaciones técnicas del dispositivo). Tenga en cuenta que el Terminal TX es el pin 1 y el terminal GND es el pin 3.

3.3. Envío de caracteres ASCII a través del terminal TX

El siguiente programa le permitirá enviar caracteres ASCII a través del terminal TX.

rs232.py

```
1 import machine
2 import utime
3 from machine import Pin, UART
4
5 led = machine.Pin("LED", machine.Pin.OUT)
6 uart = UART(0, baudrate=9600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))
7
8 while True:
9     led.on()
10    uart.write("U")
11    utime.sleep(1)
12    led.off()
13    utime.sleep(1)
```

Ejecute el programa y mida en el osciloscopio la señal recibida. Configure adecuadamente el disparo (trigger) del osciloscopio para que se capture la señal. Mida el tiempo de bit y verifique si corresponde a la medida teórica esperada.

3.4. Medida de los tiempos de bit

Envíe ahora el carácter ASCII "Y". Mida los tiempos de bit y complete los datos en la tabla sugerida. Capture las imágenes con el osciloscopio que les permita documentar esta prueba. Compruebe que $t_b = 1/\text{tasa de baudios}$. Calcule los porcentajes de error entre los datos teóricos y experimentales.

Table 1: Medición de tiempos de bit en una señal RS232.

Baudios	Tiempo de bit (Teórico)	Tiempo de bit (experimental)	Δt	% error	Voltaje máximo	Voltaje mínimo
1200						
4800						
9600						
38400						
57600						
115200						

3.5. Análisis de la estructura del protocolo RS232

Envíe cinco caracteres ASCII diferentes al carácter empleado en el numeral anterior. Use diferentes combinaciones de parámetros de la comunicación (baudios, tipo de paridad, número de bits de datos). Llene la tabla 2 con los caracteres ASCII representados en decimal y binario. Realice un dibujo hecho a mano para cada carácter. Identifique en el dibujo: bit de inicio, bits de datos, bit de paridad, bit de parada. Para cada gráfica identifique los bits LSI y MSI en la palabra de datos. Documente también cada uno de los caracteres realizando la captura correspondiente con el osciloscopio digital.

Table 2: Análisis de la trama de señales RS232.

#	Carácter ASCII	Valor Decimal	Valor Binario	Dibujo de la señal medida (Inicio, Datos, Paridad, Parada, LSB, MSB)
1				
2				
3				
4				
5				

3.6. Mediciones de la estructura del protocolo RS232

- a) Mida el tiempo total requerido para el envío de cada carácter de la tabla 2, contrástelo con lo explicado en la clase.
- b) Modifique la paridad para al menos tres de los caracteres empleados en la tabla 2. Dibuje las señales y compare las tramas. Identifique las diferencias existentes.
- c) Modifique la paridad a SIN PARIDAD y compare las tramas. Identifique las diferencias existentes.

3.7. Medida del tiempo de la trama

- a) Modifique el programa para que envíe una trama ASCII de 60 caracteres enviados a 600 baudios, paridad par y 8 bits de datos. Capture la señal resultante en el osciloscopio y mida el tiempo total de la trama generada. Compare el resultado teórico con el experimental.
- b) Realice la misma prueba anterior, pero elimine el bit de paridad. Capture la señal resultante en el osciloscopio y mida el tiempo total de la trama generada. Compare con el resultado anterior.
- c) Capture la trama "UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES" enviada a 57600 baudios, 7 bits de datos, paridad par y dos bits de parada. Mida el tiempo total del mensaje. Contraste los valores teóricos contra los experimentales.

4. RETO DE PROGRAMACIÓN

- a) **Interconexión de Hardware:** Interconecte dos dispositivos Raspberry Pi Pico cruzando las líneas de transmisión y recepción (el pin TX del dispositivo 1 al pin RX del dispositivo 2, y viceversa). Es estrictamente necesario unificar la señal GND entre ambos circuitos para establecer un nivel de referencia de voltaje común. Adicionalmente, integre una pantalla (por ejemplo, un display OLED SSD1306 o LCD 16x2 vía I2C) en el circuito para la visualización local de los datos. Conecte los dos canales del osciloscopio digital a las líneas de datos TX y RX.
- b) **Transmisión y recepción de caracteres (Half-Duplex):** Diseñe un programa de transmisión (TX.py) y uno de recepción (RX.py) para ejecutarse en los microcontroladores bajo la siguiente lógica de operación:
 - **Transmisor (TX.py):** Utilice la instrucción `input()` en la consola para preguntar al usuario el carácter o código específico que desea enviar. **El sistema debe capturar el dato exclusivamente por esta vía; no lo debe leer mediante estados lógicos en los pines físicos del dispositivo.**
 - **Receptor (RX.py):** Al detectar y decodificar el carácter, el dispositivo debe imprimir el carácter recibido en la pantalla local, y simultáneamente encender un LED indicador durante cinco segundos. Inmediatamente después, debe enviar un carácter de confirmación (ACK) de vuelta al transmisor.
 - **Confirmación:** Cuando el dispositivo transmisor reciba el carácter de respuesta, encenderá un LED de forma intermitente (parpadeo) durante tres segundos, indicando el éxito del enlace. El proceso se repite de forma permanente solicitando un nuevo ingreso de datos por consola.

- c) **Transmisión de archivos:** Cree un programa llamado *generadortxt.py* para un microcontrolador transmisor (TX) con todos los parámetros de la comunicación configurables. El programa debe preguntar al usuario el número de líneas de alfabetos en inglés que debe crear (entre 1 y 1000). El programa crea el archivo *alfabetos.txt* y enumera cada una de las líneas.

```
0001 ABCDE ..XYZ
0002 ABCDE ..XYZ
...
1000 ABCDE ..XYZ
```

Otro programa llamado *enviar.py* debe preguntar al usuario el tiempo del intervalo de envío entre líneas. Una vez se reciba la respuesta del usuario, deberá iniciar el envío línea por línea del archivo *alfabetos.txt*.

En el otro microcontrolador (RX) se deben recibir línea a línea y guardar el resultado en el archivo *recibido.txt*

Analice varios casos de envíos y de tiempos requeridos para la transferencia. ¿Cómo se pueden mejorar los programas para medir el tiempo empleado en la transferencia completa del archivo ?

¿Qué pasa si el intervalo entre líneas en el programa *enviar.py* es cero ?

5. PRELABORATORIO

5.1. Raspberry pi pico 2W

Dibuje el microcontrolador Raspberry Pi Pico 2W con sus respectivos pines. Identifique cuántos puertos UART, I2C y SPI tiene disponibles y a cuáles terminales corresponden.

5.2. Parámetros de la UART en Micropython

Consulte en la documentación oficial de Micropython: <https://docs.micropython.org/en/latest/> La configuración de la UART del dispositivo: <https://docs.micropython.org/en/latest/library/machine.UART.html>

Responda:

- ¿Cuál es la clase disponible en Micropython para la comunicación serial RS 232?
- ¿Cuáles son los parámetros configurables de la UART y qué valores pueden tomar ?
- Cree una tabla con los métodos disponibles para la clase UART. Explique cada uno de ellos.
- ¿Cómo se modifica la tasa de baudios?